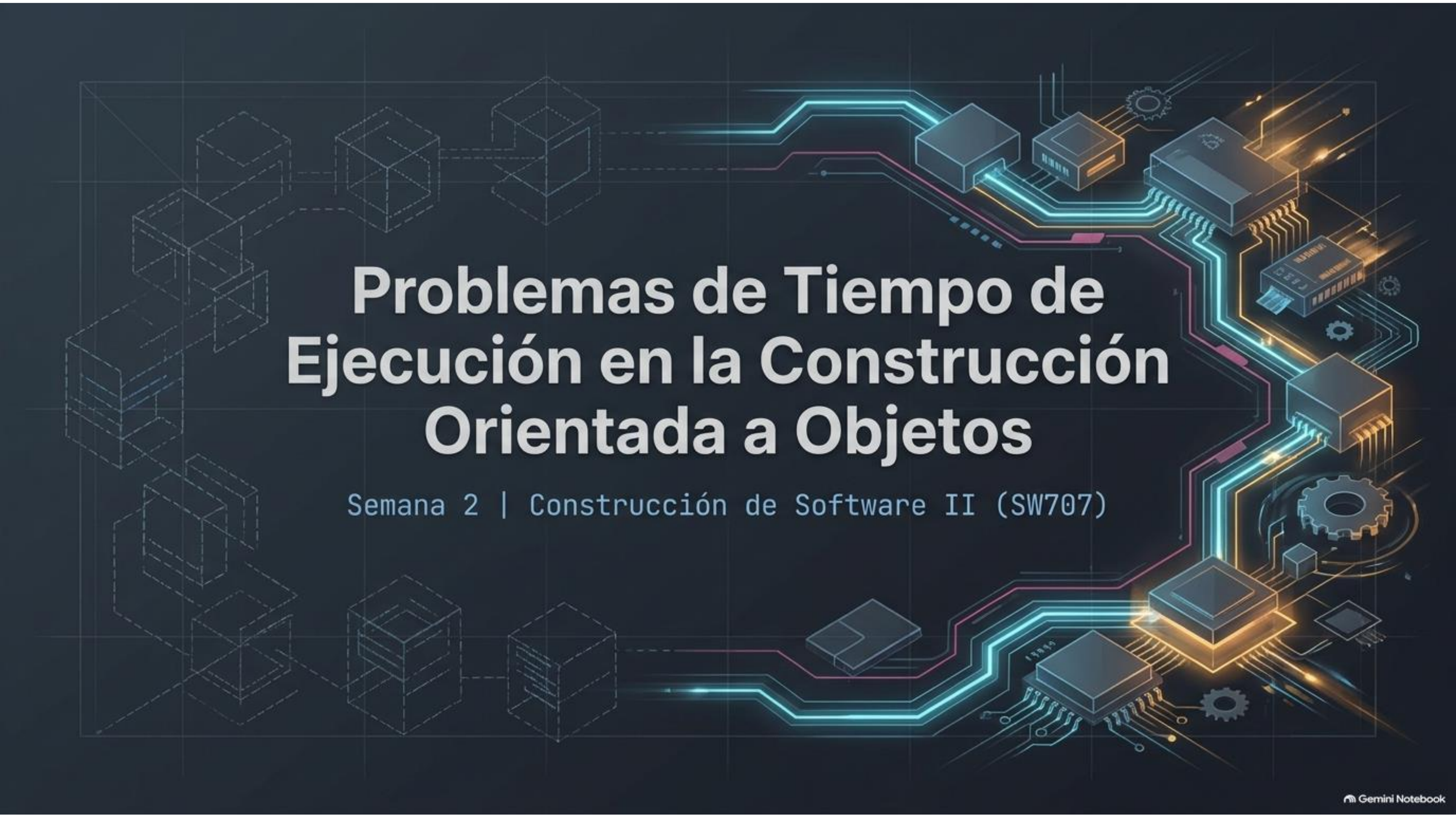


Gemini Notebook

El código en disco es estático. En memoria, está vivo.

El Plano: Lo que escribimos

```
class OrderProcessor {
    private DatabaseConnection db;
    private PaymentService payment;

    OrderProcessor(DatabaseConnection db, PaymentService payment) {
        this.db = db;
        this.payment = payment;
    }

    void processOrder(Order order) {
        if (payment.authorize(order)) {
            db.save(order);
            this.sendConfirmation(order);
        }
    }

    private void sendConfirmation(Order order) {
        // Implementation for confirmation
    }
}
```

La Maquinaria: Lo que sucede



La construcción de software moderna difiere las decisiones arquitectónicas clave hasta el último milisegundo posible. El tiempo de ejecución no es solo donde el código opera; es donde asume riesgos, consume recursos físicos y revela vulnerabilidades.

Gemini Notebook

El Continuo de la Flexibilidad



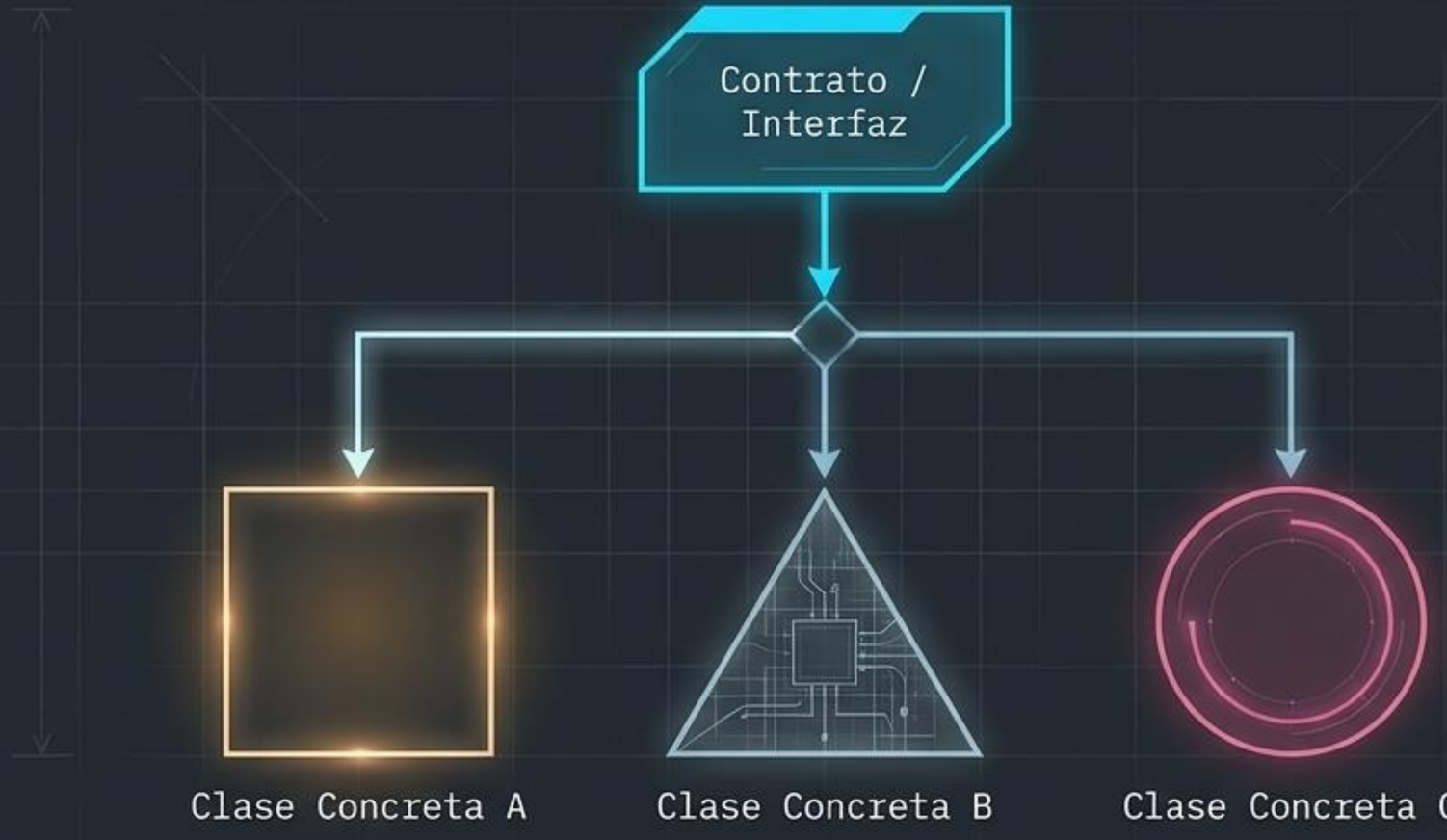
Compile-Time
(Predecible,
Estático)

Run-Time
(Dinámico,
Incierto)

A mayor delegación de decisiones al runtime, mayor poder arquitectónico, pero mayor costo computacional y superficie de ataque.

Gemini Notebook

Polimorfismo: Una interfaz, múltiples realidades



Separación del Qué y el Cómo. El sistema cliente emite un comando genérico; el runtime decide qué maquinaria específica ejecutar basándose en el tipo real del objeto instanciado, permitiendo extender el sistema sin modificar el código consumidor.

Gemini Notebook

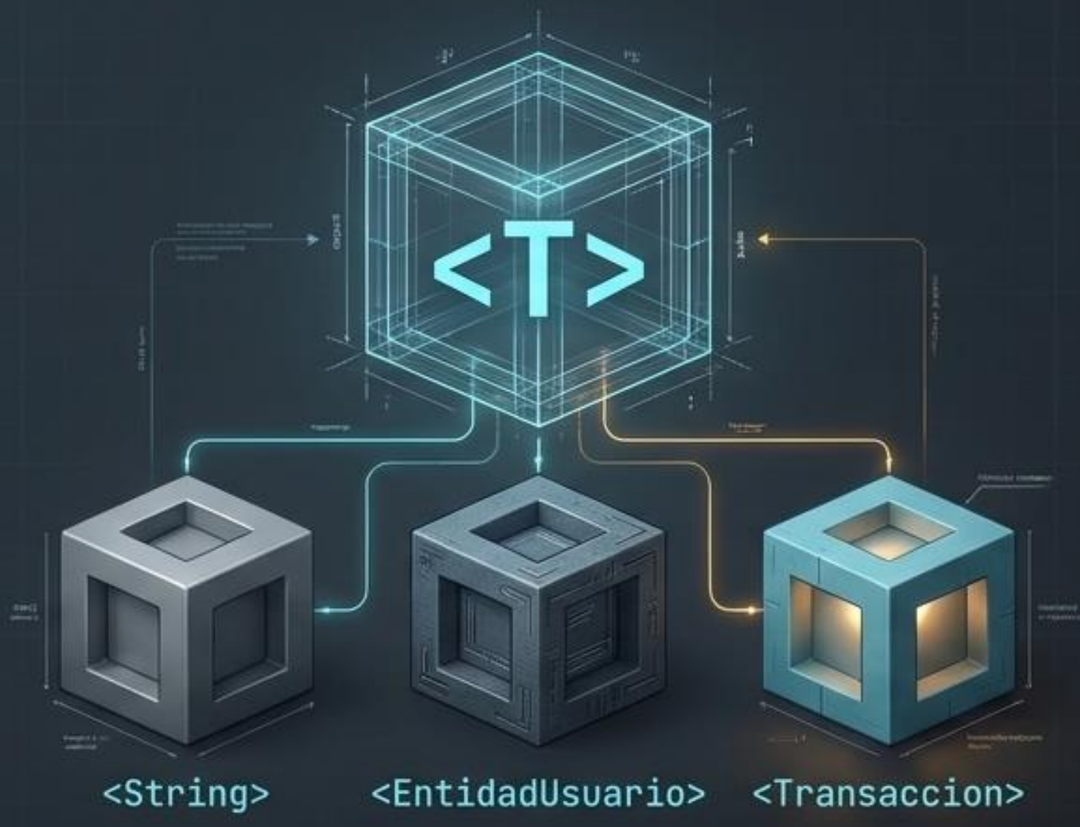
El Mecanismo: Enlace Dinámico (*Dynamic Binding*)



El polimorfismo no es magia; es una indirección de memoria. El compilador no puede fijar la dirección de la función. En su lugar, el runtime debe consultar una tabla de métodos virtuales (V-Table) en plena ejecución para resolver la dirección exacta, incurriendo en un salto dinámico.

Gemini Notebook

Genéricos y Plantillas: Composición Estructural



Polimorfismo paramétrico. Permite reutilizar la estructura lógica y el comportamiento de un componente garantizando la seguridad de tipos estática. Elimina la necesidad de conversiones (casting) forzadas y propensas a errores durante el tiempo de ejecución.

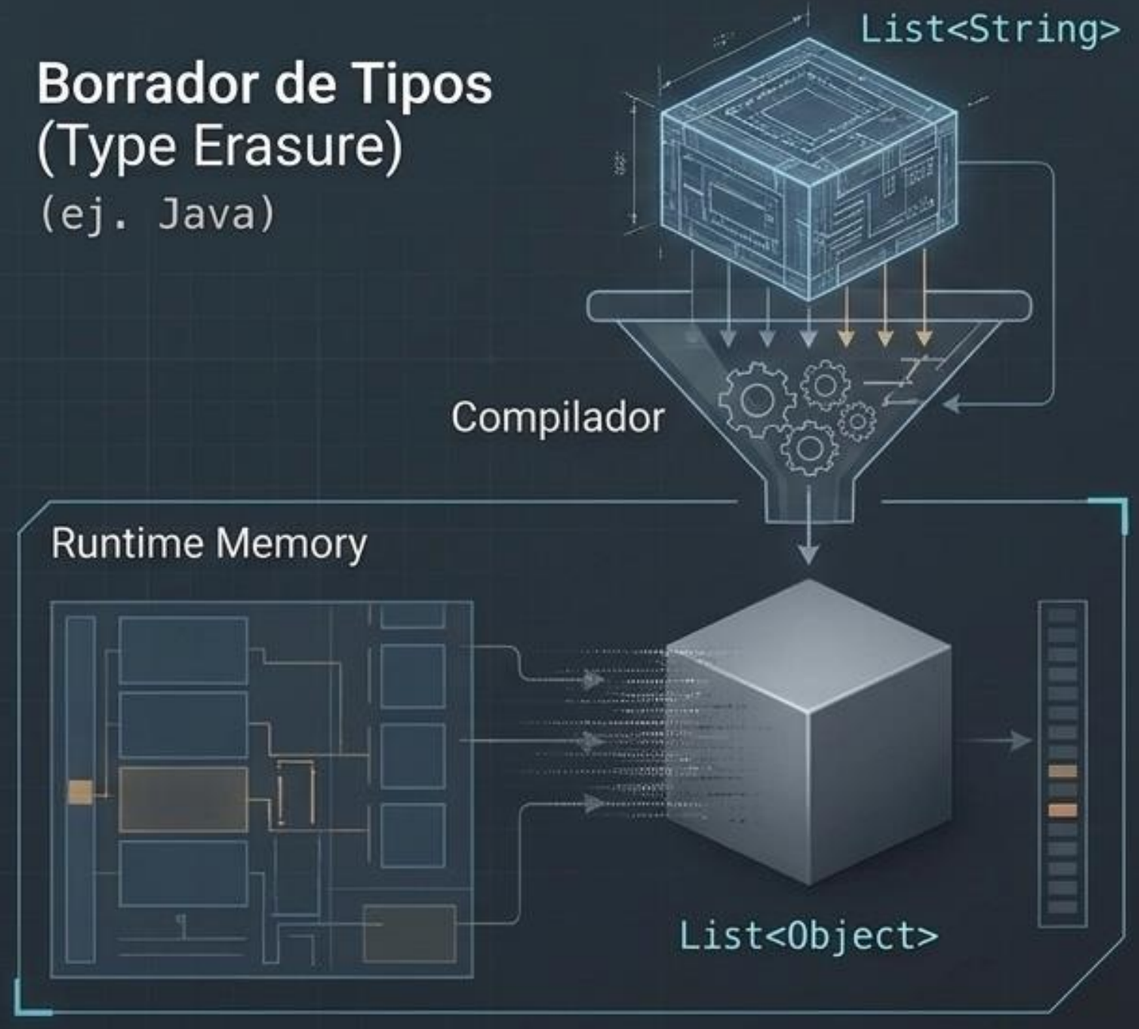
Gemini Notebook

Ilusión Estática vs. Realidad de Memoria

Cómo los diferentes runtimes gestionan la parametrización

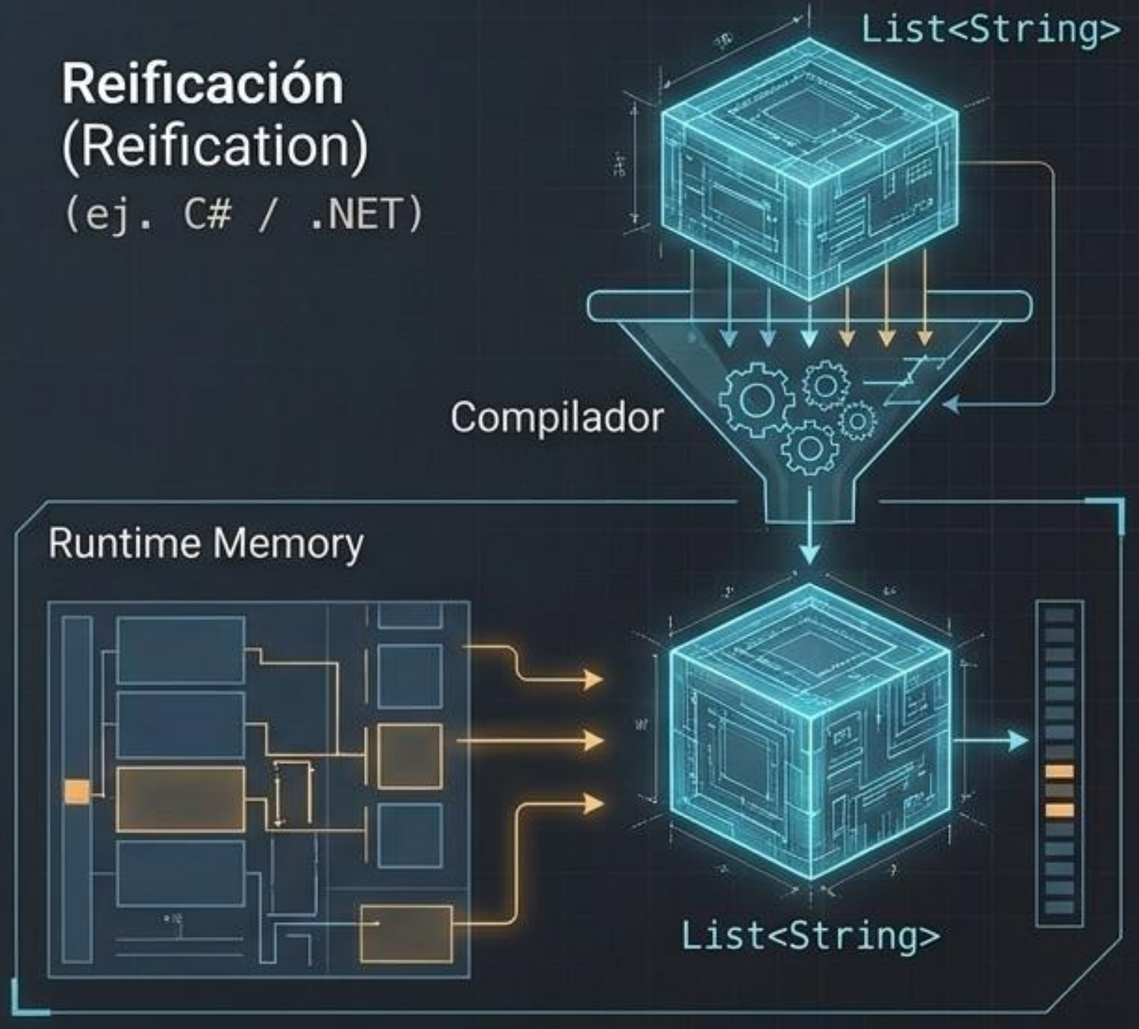
Borrador de Tipos (Type Erasure)

(ej. Java)



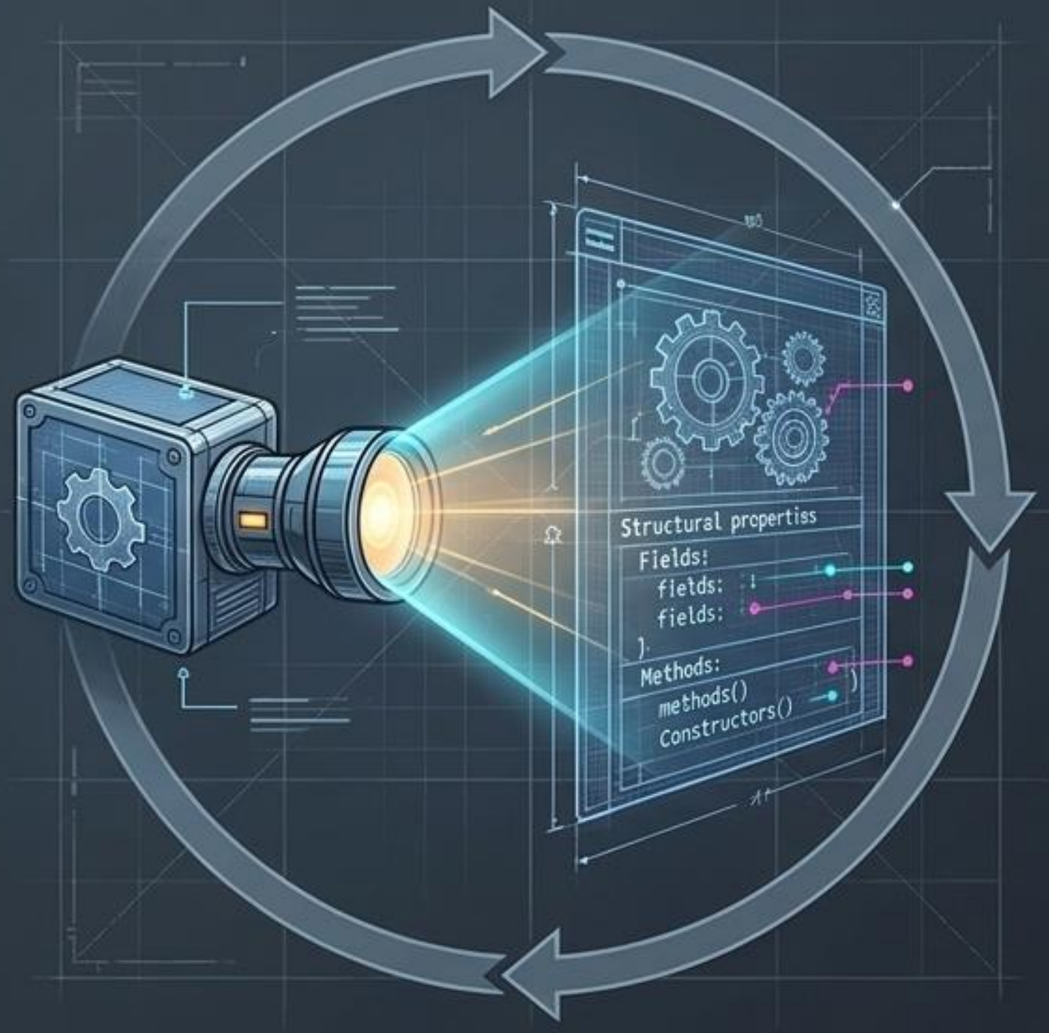
Reificación (Reification)

(ej. C# / .NET)



Gemini Notebook

Reflexión y Metaprogramación: Código que inspecciona código



Inspección:

Capacidad de leer la metadata de la clase dinámicamente (descubrir campos, métodos, constructores).

Invocación:

Capacidad de ejecutar métodos o instanciar objetos pasando sus nombres como cadenas de texto (`strings`) en pleno vuelo.

Contexto en la Industria:

Es el motor subyacente detrás de frameworks modernos (`Spring`, `Hibernate`, .NET Core) para mapeo relacional e inyección de dependencias.

Gemini Notebook

Carga y Enlace Tardío: Arquitecturas Extensibles



El Concepto Central:

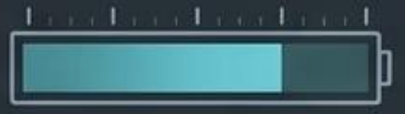
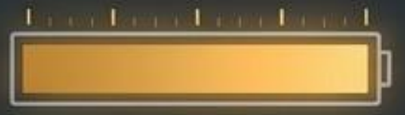
La capacidad de aplazar la resolución y carga de componentes hasta que (y solo si) son explícitamente requeridos por el sistema, sin necesidad de recompilar la aplicación central.

Casos de Uso Arquitectónicos:

- Inyección de dependencias (DI Container)
- Sistemas basados en plugins modulares
- Configuración resoluble en tiempo de ejecución (Semana 11)

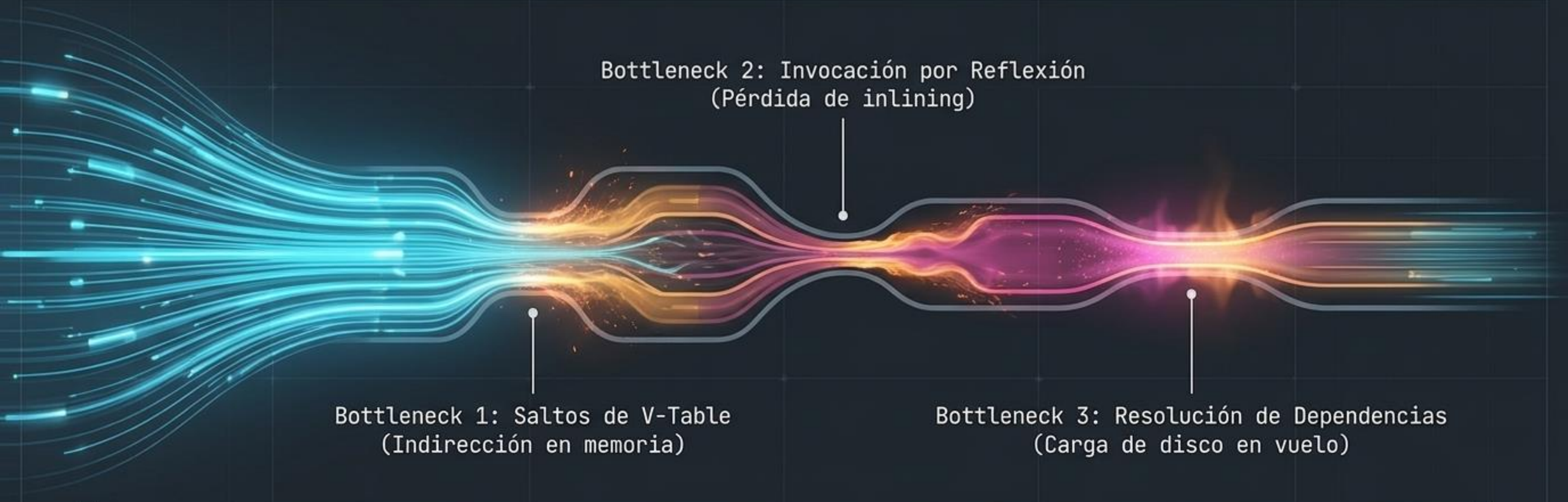
Gemini Notebook

Diagnóstico Comparativo de Mecanismos de Runtime

	Flexibilidad Dinámica	Costo de Rendimiento	Superficie de Riesgo	Caso de Uso Típico
Genéricos	 LOW	 ZERO/NEGLIGIBLE	 ZERO	Estructuras de datos seguras
Polimorfismo	 MEDIUM-HIGH	 LOW	 LOW	Desacoplamiento de lógica
Carga Tardía	 HIGH	 MEDIUM	 MEDIUM	Sistemas de Plugins modulares
Reflexión	 MAXIMUM	 HIGH	 HIGH	Frameworks y Mapeo ORM

Gemini Notebook

La Factura Oculta: El Costo del Rendimiento



Implicancia Arquitectónica: El dinamismo extremo desactiva optimizaciones críticas del compilador (como el inlining de métodos) y genera 'cache misses' en el procesador debido a los saltos impredecibles de memoria.

Gemini Notebook

La Superficie de Ataque: Cuando la Flexibilidad es Vulnerabilidad



Inyección de Dependencias Maliciosas

Ejecución Remota de Código (RCE) vía Reflexión

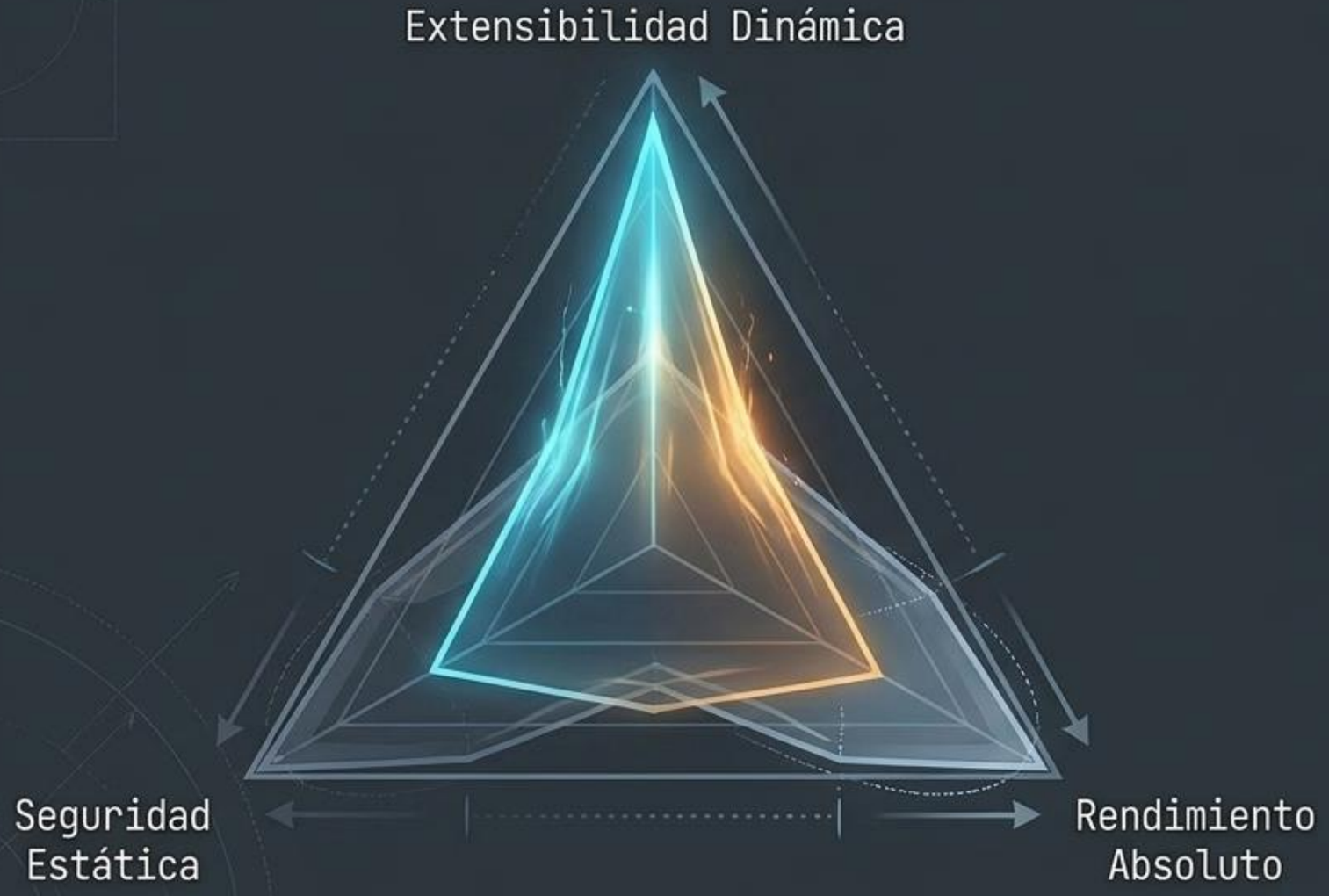
Deserialización Insegura de Objetos

Principio de Seguridad: Si el sistema puede cargar componenttes o invocar métodos basándose en cadenas de texto externas durante la ejecución, un atacante puede manipular dichas entradas.

Caso Histórico: Log4Shell – El ejemplo definitivo de fallo catastrófico causado por características dinámicas resolviendo strings externos (JNDI lookup) en tiempo de ejecución.

Gemini Notebook

El Dilema del Ingeniero: El Triángulo de Trade-offs



Principio Arquitectónico:

No existen soluciones perfectas, solo compromisos. Maximizar la flexibilidad dinámica mediante reflexión o enlace tardío reduce inevitablemente la superficie de rendimiento garantizado y la seguridad estática del sistema. El objetivo es calibrar el balance exacto para cada módulo.

Gemini Notebook

Hacia la Construcción Robusta: Domesticando el Runtime



El Desafío: Dado que el entorno de ejecución
Dado que el entorno de ejecución es inherentemente impredecible, costoso y peligroso, el software debe ser diseñado para protegerse a sí mismo.

Próximas Semanas:

- Semana 3: Desarrollo por Contratos (Precondiciones, Postcondiciones, Invariantes).
- Semana 4: Programación Defensiva y Barricadas de Validación.

Gemini Notebook

La construcción de software no termina en el compilador.

Diseña para la flexibilidad.
Construye para el rendimiento.
Defiéndete contra la incertidumbre del runtime.

Gemini Notebook